LabelLens: Implementation Plan (v2)

AI Food Label Intelligence & Packaging Compliance Platform, India edition

Status: plan only. Nothing is implemented. The repository is empty (greenfield). This version applies your decisions: India only, English only, Gemini for OCR, zero per-scan budget, no guests, three roles.

1. Product scope and user journeys

Role	What they can do
Customer	Scan a pack (front, ingredients, nutrition, declarations), review and correct fields, see plain-language nutrition, ask questions in chat, view history, compare products (including with international counterparts)
Manufacturer	Upload a proposed label, run a pre-print checklist against the active rule set, keep versions of drafts, export a preliminary report
Inspector	Scan packs in the field, review OCR evidence, run compliance checks with traceable findings, export reports. **Only role that can edit rules**, and only after re-entering the admin password (step-up); every change is logged

No guest mode: all use requires sign-in. There is no separate Admin role; the inspector's rule-governance powers replace it (see decision A in section 8).

Core flow: sign in → role workspace → capture panels → image quality check → Gemini extraction → review/correct → deterministic nutrition → deterministic rules (inspector/manufacturer) → results → chat → save/export.

International comparison: a customer or inspector picks a scanned Indian product and a counterpart from **Open Food Facts** (open, free database) and sees side-by-side facts per 100 g (ingredients, sugar, sodium, additives). Output is *factual differences only*, never a "bad product" verdict, because labelled accusations about a named company carry defamation risk and the data may differ by recipe, year or database error.

2. Recommended architecture and why

Best fit for you: Next.js (TypeScript) + Supabase (Postgres, Auth, private Storage) + Gemini API, hosted on free tiers.

Option	Cost	Verdict
Next.js + Supabase + Gemini free tier	₹0 to start; one language; auth/DB/storage in one service; row-level security for permissions	**Chosen**
Next.js + FastAPI + self-hosted Postgres	Free but you operate two services and a server	Only if you need offline/on-prem OCR
Flutter / React Native	Better camera and offline	Later; a mobile-friendly web app (PWA) first

Why zero budget changes the OCR choice: Gemini's free tier is the cheapest option, but (1) its rate limits are small and can change, so scanning goes through a **queue with per-user limits**; (2) on free tiers, Google's terms may allow submitted content to be used to improve its products, so uploads must carry a clear privacy notice and no personal photos; (3) free quotas are not a production guarantee. **Fallback:** Tesseract OCR (free, self-run) behind the same `OcrProvider` interface, plus manual entry, so the app still works when quota runs out. *All free-tier terms and limits must be re-checked on the provider's current pages before launch; I have not verified them.*

Layout: `apps/web` (UI and server routes), `packages/core` (nutrition and rules engine, pure functions, no I/O), `packages/rules` (versioned rule data), `db/migrations`, `docs/`, `.env.example`.

3. Component responsibilities and data flow

Flow: *Browser → server route (authorization) → Storage → extraction job → DB → review UI → nutrition → rules → report/chat.*

Component	Does	Input → output	Depends on	Tested by
Web app (Next.js PWA)	Camera/upload, crop/rotate, review, results	Images, edits → API calls	API	Component tests, Playwright
Auth + roles (Supabase Auth)	Sign-in, role claims, step-up re-auth for rule edits	Credentials → session with role	Supabase	Authorization matrix tests
API layer	Enforces permissions on every call; row-level security in DB	HTTP → JSON	Auth, DB	API authz and integration tests
Storage	Private bucket, signed URLs	Image → asset ID	Supabase	Upload validation tests
Preprocessing (sharp)	Rotate, downscale, blur/brightness warnings	Image → cleaned image + quality score	sharp	Fixture images
Gemini extraction	Reads text, returns structured JSON with region and confidence	Image(s) → candidate fields	Gemini key (server only)	Golden-set fixtures, stubbed provider in CI
Validation (Zod)	Rejects malformed output, normalizes units, flags contradictions	Candidates → `ExtractedField` rows (ok/uncertain/missing)	core	Mixed-unit and contradiction tests
Nutrition module	Per-100 g ↔ serving ↔ pack; sugar → teaspoons; sodium vs salt	Fields → values + assumptions, or “cannot compute”	none	Unit/property tests
Rules engine	Evaluates versioned rules	Fields + rule-set version → findings	Rule files in DB	Per-rule test tables
Comparison	Fetches the Open Food Facts product and aligns units	Barcode/name → comparison table	Open Food Facts API	Mocked API tests
Chat	Grounded answers with field citations	Question + product record → answer	Gemini	Groundedness and refusal tests
Reports	Preliminary PDF/CSV with provenance	Run → file	Findings	Snapshot tests
Ops	Structured logs (no images or PII), CI, error tracking	n/a	Free tiers	Smoke tests

4. Database and API outline

Entities: User(role), Scan(owner, type: product | draft_label, status), ImageAsset, ExtractedField (value, unit, confidence, status, source image, bbox), FieldEdit, RuleSet(version, status, effective_from), ComplianceRule, ComplianceRun, ComplianceFinding, ComparisonResult, ChatSession, ChatMessage, **AuditEvent (append-only)**. Product catalogue and an Ingredient table are deferred.

Ownership: customers and manufacturers see only their own records; inspectors see the scans they create or are assigned; nobody can edit or delete audit events.

Scan status: uploaded → processing → needs_review → reviewed → completed; failed (retryable).

Endpoint	Purpose	Roles
`POST /api/scans`, `GET /api/scans/:id`	Create/read	all (owner-scoped)
`POST /api/scans/:id/images`, `/extract`	Upload, start extraction	owner
`PATCH /api/scans/:id/fields/:fid`	Correct a field (audited)	owner

Endpoint	Purpose	Roles
`POST /api/scans/:id/compliance-runs`	Run rules	inspector, manufacturer
`POST /api/scans/:id/compare`	International comparison	customer, inspector
`POST /api/scans/:id/chat`	Ask a question	owner
`POST /api/rulesets`, `PATCH /api/rules/:id`	Draft/publish rules (needs step-up)	inspector
`GET /api/audit`	View audit log (read-only)	inspector
`GET /api/me/export`, `DELETE /api/me`	User data export/deletion	self

5. AI, OCR, nutrition and compliance safety boundaries

- **Extraction:** unreadable or absent values are stored as missing/uncertain and never invented. Model confidence is a hint, not a measured probability.
- **Nutrition (deterministic):** sugar → teaspoons only from an explicit sugar value with the assumption stated; per-pack only when net quantity and units support it; total vs added sugar, sodium vs salt, saturated vs trans fat stay separate; oils named only if printed; ingredient order never turned into percentages; no healthy/unhealthy verdicts.
- **Compliance (deterministic, versioned):** each rule has a stable ID, legal source, applicability, required fields, validation logic, effective date, severity and explanation. Results: PASS / FAIL / NEEDS REVIEW / NOT APPLICABLE. Low confidence always yields NEEDS REVIEW. Detecting text does not prove compliance; font size, placement and category exceptions need measured or human review. Reports say “preliminary assisted check, not certification or legal advice”.
- **Rule governance (your requirement):** only inspectors edit rules; each edit requires step-up with the admin password, creates a **new draft version** (published versions are immutable), and logs who, what, before/after and when. *My recommendation:* require a second inspector to approve publication (“four-eyes”), and store admin credentials hashed with a strong algorithm, never shared in plain text. Past reports stay tied to the rule version they used.
- **Legal sources:** Legal Metrology (Packaged Commodities) Rules, 2011 and amendments; FSSAI labelling regulations as a separate later domain. I will not quote rule numbers or thresholds from memory: the inspector enters each rule from the verified official text.
- **Chat:** receives the product record, calculations and a small approved reference set; labels each statement as label fact, calculation, general knowledge or unknown; cites field IDs; refuses medical/legal advice; cannot run or alter compliance. Sessions are scoped to (user, scan).

6. Security, testing, deployment, cost

- **Security:** permissions enforced in the backend and by database row-level security; secrets in environment variables only (`.env.example` committed, no real keys); private storage with signed URLs; upload type/size limits and EXIF stripping; rate limits and quota queue; safe error messages; export/delete of user data.
- **Testing:** unit tests (nutrition, every rule with pass/fail/needs-review cases); validation tests; authz matrix per role and endpoint; integration tests with a stub OCR; Playwright journeys (customer scan → chat; correction audit trail; manufacturer pre-check; inspector rule edit with step-up; unauthorized access). Edge cases: blurry images, missing nutrition panel, mixed units, per-serving vs per-100 g, unreadable MRP/date, contradictory OCR, allergens, low confidence, unsupported packaging, network failure (retry message only; offline is not promised).
- **Deployment (free tiers):** Vercel or Cloudflare Pages for the app, Supabase free project for DB/auth/storage, GitHub Actions CI. Free tiers pause or limit usage, so this suits a pilot, not scale. Data residency: free-tier region may not be in India; decide if this matters.
- **Cost:** ₹0 only while inside free quotas. If quota is exceeded, scans queue or fall back to Tesseract/manual entry. Actual quotas and latency to be measured.

7. Phased roadmap (build everything, in order)

1. **Foundation:** repo, CI, schema, auth, three roles, audit log, storage, one thin scan flow with a stub OCR.
2. **Customer scan:** capture, quality checks, Gemini extraction, review/correct with audit trail, queue and fallback.
3. **Nutrition and explanations:** calculation module and plain-language UI.
4. **Chat:** grounded assistant with citations.

5. **Compliance and inspector:** rules engine, versioned rule sets, step-up editing, findings UI, reports.
6. **Manufacturer:** draft-label scans, pre-print checklist, versions.
7. **International comparison:** Open Food Facts integration, factual comparison view.
8. **Hardening:** security review, accessibility, privacy notice, pilot with real packs.

Acceptance: every value traceable to an image region; no computed value without its inputs; rules pass their test tables; permission matrix passes; inspectors have verified each active rule against official text; reports carry the disclaimer. OCR accuracy is measured on a labelled sample of real Indian packs, not guaranteed.

8. Assumptions, risks, and remaining decisions

Assumptions: India only, English only; free tiers throughout; mobile-friendly web app first.

Risks: Gemini free-tier limits/terms; OCR on curved or glossy packs; rules entered by hand can be wrong (mitigated by versioning, test tables, four-eyes); Open Food Facts coverage of Indian and foreign products is uneven; comparisons could be misread as accusations.

Please decide:

A. Admin: keep **no separate admin**, with inspectors editing rules via step-up password (my reading of your message), or add a small admin role? I recommend four-eyes approval either way. B. Accept that free Gemini may use uploads for model improvement, or use Tesseract-only? C. Do you have sample pack images and who is the first inspector/reviewer? D. Which product categories first (e.g., biscuits, beverages, snacks)?

9. Demo setup (two phones + laptops, live sync)

Target demo: Phone 1 = customer, Phone 2 = manufacturer (both take photos and upload). Laptop A = customer dashboard, Laptop B = manufacturer dashboard, Laptop C = inspector view. Everything updates live, with no refresh.

Piece	How it works
Devices	One web app opened in each browser, no install. Each device signs in with its own role account.
Live sync	Supabase Realtime pushes database changes (new scan, status change, extracted field, correction, finding) to every open dashboard within about a second or two.
Flow on screen	Upload → `uploaded` → `processing` → `needs_review` → fields appear → inspector feed shows the new scan live.
Hosting	Deploy to a free host (e.g. Vercel) so all devices use one HTTPS link. Browser cameras need HTTPS; plain `http://laptop-ip` only offers the photo picker. A local run needs an HTTPS tunnel.
Internet	All devices need working internet (Gemini and Supabase are cloud services).

Build order for the demo (thin end-to-end loop first):

1. Sign-in, three roles and live dashboards per role.
2. Phone photo capture/upload appearing live on the dashboards.
3. Gemini extraction with Tesseract/manual fallback, then field review.
4. Inspector live feed with a small seeded rule set.

Then deepen: nutrition, chat, rule editing with admin password, comparison, reports.

Demo risks and mitigations

- **Gemini free quota or rate limit mid-demo:** cache results and add a “use sample extraction” fallback so the demo cannot stall.
- **Wi-Fi failure:** have a phone hotspot ready as backup.
- **Placeholder rules:** demo rules are clearly labelled *placeholder, not legal fact*. The inspector enters real rules from the official text before any real use.
- **Test packs:** prepare 3-5 real Indian packs and their photos in advance.